

MST BLOCKCHAIN

MCP Server

Technical Documentation & Developer Overview

Version 1.0 • September 2026
mstblockchain.com

Overview: MST Blockchain MCP Server

The MST Blockchain MCP (Model Context Protocol) Server is an intelligent documentation gateway that allows AI assistants — including Claude — to access, search, and surface the complete MST blockchain knowledge base in real-time.

By bridging natural-language AI interactions with structured blockchain documentation, the mst-mcp server eliminates the need for developers to manually search through docs, enabling faster learning curves and more productive development sessions.

What is MCP?

Model Context Protocol (MCP) is an open standard introduced by Anthropic that enables AI models to connect securely to external data sources and tools. It defines a standard interface between an AI assistant and external "MCP Servers" that expose resources, tools, and prompts.

Key Benefit Instead of static, pre-baked knowledge, an MCP server gives an AI assistant live, structured access to authoritative documentation.

MST Blockchain: Quick Overview

MST Chain is an EVM-compatible Layer-1 blockchain designed for high-throughput enterprise and developer use. Key characteristics include:

- EVM & Solidity compatibility — deploy existing Ethereum contracts without modification
- Native gas token (MSTC) with structured fee markets
- Validator-based consensus with a defined network & staking architecture
- Full support for Mainnet and Testnet deployment pipelines
- Rich SDK and IDE tooling for Web2→Web3 developer onboarding

The mst-mcp Server

The mst-mcp server acts as the official documentation interface for MST Blockchain. It is deployed at:

<https://mcp.mstblockchain.com/sse>

It exposes three core tools:

Tool	Description	Use Case
list_documents	Returns all available documentation filenames	Discover what topics are available
read_document	Reads and returns full content of a named file	Deep-dive into a specific topic

Tool	Description	Use Case
search_documents	Full-text search across all documents	Find answers across the knowledge base

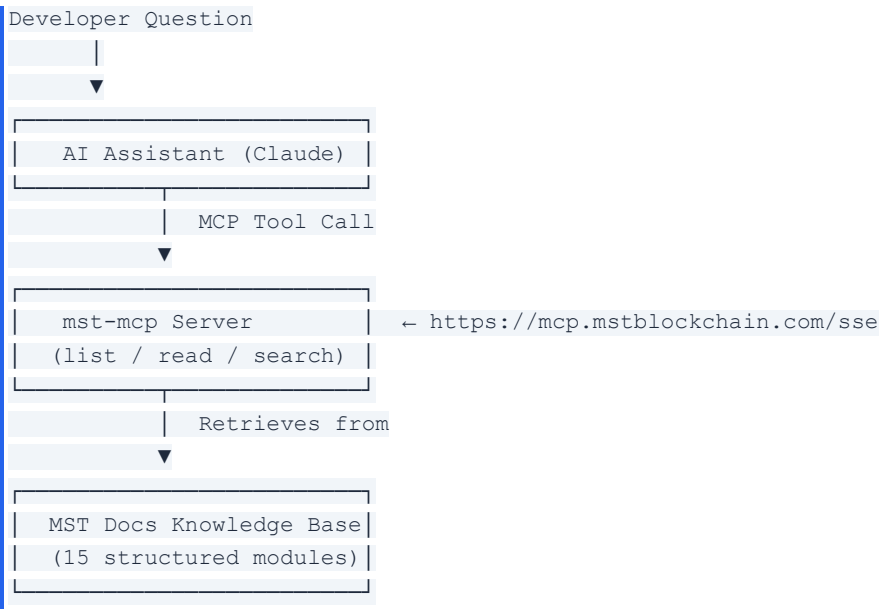
Knowledge Base Coverage

The mst-mcp server currently indexes the following documentation modules:

- Introduction to MSTC — overview of the native token and its role
- MST Chain Introduction — architecture and philosophy
- Network & Validator Structure — node types, staking, consensus
- Accounts on MST Chain — externally owned accounts, contract accounts, key management
- Transactions — lifecycle, signing, broadcasting, receipts
- Blocks — block structure, block time, finality
- Gas — fee model, gas estimation, optimization
- Introduction of EVM & OpCodes — compatibility layer, opcode reference
- Solidity Smart Contract Programming — language features, patterns, best practices
- MST Blockchain SDK — JavaScript/TypeScript SDK reference
- SDK (supplemental) — additional SDK examples and API coverage
- IDE & Hands-on Practices — toolchain setup, Hardhat/Foundry integration
- Development Networks / MST Testnet — testnet endpoints, faucet, configuration
- Deployment on Mainnet — production deployment checklist and procedures
- Web2 and Web3 — conceptual bridge for traditional developers

Integration Diagram

Below is the high-level interaction flow when an AI assistant uses the mst-mcp server:



Who Should Use This

Audience	How mst-mcp Helps
Solidity Developers	Get instant answers on MST EVM differences, gas optimization, and contract deployment
Backend / Web2 Developers	Bridge Web2 → Web3 concepts and learn MST SDK integration
DevOps / Infrastructure	Understand node setup, validator structure, and network config
Product & Technical Writers	Quickly reference authoritative MST documentation for accurate content
AI Assistant Users	Ask blockchain questions in plain English; the MCP returns precise answers

Technical Documentation: mst-mcp Server

1. Connection & Protocol

The mst-mcp server implements the Model Context Protocol over Server-Sent Events (SSE). This transport layer is ideal for streaming tool results to AI clients.

Parameter	Value
Server Name	mst-mcp
Transport	SSE (Server-Sent Events)
Endpoint URL	https://mcp.mstblockchain.com/sse
MCP Version	Latest stable
Authentication	None (public read-only server)
Rate Limiting	Standard fair-use limits apply

2. Tool Reference

2.1 list_documents

Returns a list of all documentation filenames available in the knowledge base. Use this as a first step to discover what topics are indexed.

Parameters:

- None — takes no arguments

Response:

A newline-separated or structured list of filenames, e.g.:

```
Transactions.txt
Network & Validator Structure.txt
Introduction to MSTC.txt
SDK.txt
Gas.txt
... (15 total modules)
```

When to use:

- At the start of a session to understand what documentation is available
- Before calling read_document to confirm the exact filename

2.2 read_document

Reads and returns the complete contents of a specific documentation file. This gives full detail on a given topic.

Parameters:

Parameter	Type	Required	Description
filename	string	Yes	Exact filename as returned by list_documents (e.g. "Gas.txt")

Example Call:

```
read_document({ filename: "Gas.txt" })
```

Response:

- Full text content of the documentation file
- May include sections, sub-sections, code examples, and reference tables

When to use:

- When you need comprehensive information on a specific MST topic
- When search_documents returns a match and you want the full context

2.3 search_documents

Performs full-text keyword search across all documentation files. Returns matching lines with their filename and line number.

Parameters:

Parameter	Type	Required	Description
query	string	Yes	The keyword or phrase to search for across all documents

Example Call:

```
search_documents({ query: "gas estimation" })
```

Response:

- Matching lines from across all documentation files
- Each result includes the source filename and line context
- Returns empty result if no matches are found

When to use:

- To quickly find which document(s) cover a specific topic
- To look up a specific term, function name, or concept across the entire knowledge base
- As a first step before read_document when the relevant file is unknown

3. Connecting mst-mcp in Claude.ai

Users can connect the mst-mcp server directly in the Claude.ai interface using the MCP connectors panel:

1. Open Claude.ai in your browser
2. Navigate to Settings → Connectors (or MCP Apps)
3. Click "Add MCP Server" or "Connect New MCP"
4. Enter the server URL: `https://mcp.mstblockchain.com/sse`
5. Set the server name to "mst-mcp"
6. Authentication credentials
 OAUTH_CLIENT_ID=mst-mcp-client
 OAUTH_CLIENT_SECRET=mst-mcp-secret-key-350e3445
7. Save and activate — the three tools are now available in chat

Tip Once connected, Claude will automatically call mst-mcp tools when you ask MST blockchain questions — no special commands needed.

4. Connecting via the Anthropic API

Developers using the Anthropic API can include the mst-mcp server in their API calls using the `mcp_servers` parameter:

```
const response = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-6",
    max_tokens: 2000,
    messages: [
      { role: "user", content: "How does gas estimation work on MST?" }
    ],
    mcp_servers: [
      {
        "type": "url",
        "url": "https://mcp.mstblockchain.com/sse",
        "name": "mst-mcp"
      }
    ]
  })
});
```

5. Knowledge Base Index

The following table maps each documentation module to its key topics and recommended audience:

Document	Key Topics	Audience
Introduction to MSTC	Native token, tokenomics, utility	All
MST Chain Introduction	Architecture, philosophy, use cases	All
Network & Validator Structure	Consensus, staking, node types	DevOps, Validators
Accounts on MST Chain	EOAs, contract accounts, key management	Developers
Transactions	Signing, broadcasting, receipts, lifecycle	Developers
Blocks	Block structure, time, finality, confirmations	Developers
Gas	Fee model, gas price, estimation, optimization	Developers
EVM & OpCodes	EVM layer, opcode reference, compatibility	Advanced Devs
Solidity Smart Contracts	Language features, patterns, security	Smart Contract Devs
MST Blockchain SDK	JS/TS SDK, API reference, setup	Frontend/Backend Devs
SDK (supplemental)	Additional examples, edge cases	Developers
IDE & Hands-on Practices	Hardhat, Foundry, local toolchain	Developers
Development Networks	Testnet config, faucet, RPC endpoints	Developers
Deployment on Mainnet	Production checklist, deployment steps	DevOps, Developers
Web2 and Web3	Conceptual bridge, paradigm differences	Web2 Developers

6. Recommended Usage Patterns

Pattern A: Topic Discovery

```
1. list_documents()           → get all filenames
2. Identify the relevant file
3. read_document({ filename }) → get full content
```

Pattern B: Keyword Search

```
1. search_documents({ query }) → find matching lines
2. Note the source filename(s)
3. read_document({ filename }) → get full context
```

Pattern C: Direct Read (when file is known)

```
1. read_document({ filename: "Gas.txt" }) → immediate answer
```

7. Error Handling

Scenario	Behaviour	Recommended Action
No approval received	Tool returns an error	Ensure the MCP server is properly authorized in your client
Empty search result	Returns {}	Try different keywords or use list_documents to browse topics
File not found	Error on read_document	Call list_documents first to confirm the exact filename
Network timeout	SSE connection drops	Retry the request; the server is stateless

8. Security & Privacy

- The mst-mcp server is read-only — it does not accept writes or mutations
- No user data or query history is stored on the server
- All communication is over HTTPS (TLS encrypted)
- No API key or authentication is required for documentation access

9. Support & Resources

Resource	Link
MST Blockchain Website	https://mstblockchain.com
MCP Server Endpoint	https://mcp.mstblockchain.com/sse
Claude.ai MCP Docs	https://claude.ai/docs/claude-tag/overview
Anthropic MCP Guide	https://docs.anthropic.com